

UNIVERSITÀ DEGLI STUDI DI TORINO
Dipartimento di Fisica

**REPORT
RETI NEURALI**

RICONOSCIMENTO DI SEGNALI STRADALI

A CURA DI
MARCO GORGONE

ANNO 2025/2026

Indice

1	Problema: Riconoscimento dei Segnali Stradali	3
1.1	Dataset	4
2	Metodo	6
2.1	Rete Utilizzata	6
2.2	Funzione di Loss	6
2.3	Valutazione	9
3	Analisi	10
3.1	Scelta del Modello	10
3.2	Curve ROC	11
3.3	Calibrazione del Learning Rate e del Momento	12
3.3.1	Analisi dell'Accuratezza	12
3.3.2	Analisi della Loss	13
3.3.3	Curve ROC per le Diverse Configurazioni	14
3.3.4	Analisi dell'Addestramento Prolungato	15
3.3.5	Sintesi dei Risultati e Configurazione Ottimale	16
4	Codice	17
4.1	Caso Reale	18
4.1.1	Divisione	18
4.1.2	YOLO	18
4.1.3	Analisi dei Risultati	19
5	Conclusione	21

1 Problema: Riconoscimento dei Segnali Stradali

La progressiva diffusione dell'Intelligenza Artificiale nei sistemi di trasporto ha reso il riconoscimento automatico dei segnali stradali un tema di primaria rilevanza. Lo sviluppo di veicoli a guida autonoma richiede modelli affidabili, in grado di supportare decisioni sicure in scenari dinamici e complessi. In tale contesto, la corretta interpretazione della segnaletica rappresenta un prerequisito essenziale per robustezza e sicurezza operativa.

Nel traffico reale compaiono segnali eterogenei (ad esempio limiti di velocità, divieti, obblighi e avvisi), la cui corretta interpretazione è necessaria per una guida conforme e sicura. In modo analogo, un sistema di guida autonoma deve classificare tali segnali con elevata accuratezza, così da tradurre l'informazione visiva in decisioni operative coerenti.

In questo lavoro viene proposto un modello di classificazione dei segnali stradali basato su una rete neurale convoluzionale (CNN).

La Figura 1 riporta un esempio visivo del problema affrontato.

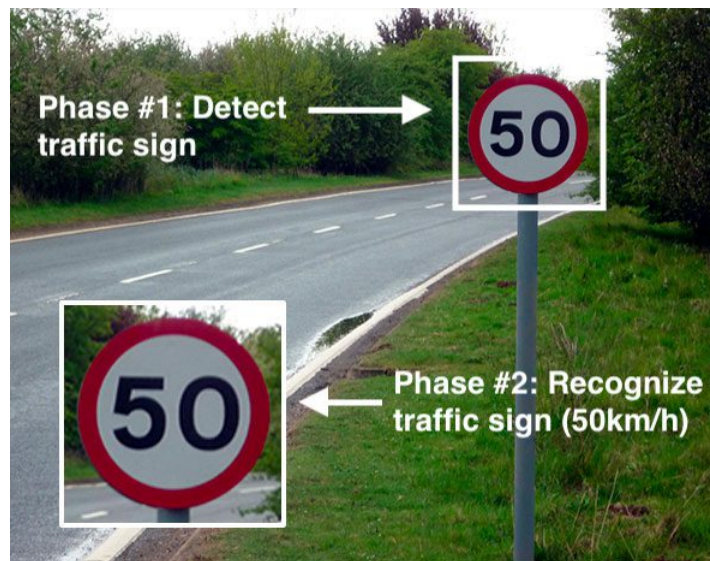
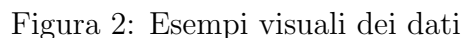


Figura 1: Esempio

Il dataset completo, disponibile all’indirizzo <https://www.kaggle.com/datasets/meowmeowmeowmeowmeows/gtsrb-german-traffic-sign>, contiene oltre 50.000 immagini di segnali stradali e occupa circa 560 MB. Nel presente lavoro è stato utilizzato il sottoinsieme presente nel percorso DITS, con dimensione complessiva pari a circa 450 MB.



- Indicazione
- Pericolo

- Divieto
- Background (utile per un secondo scopo)

2 Metodo

In questa sezione viene descritto l'approccio adottato per risolvere il problema, formulato come task di classificazione in quattro categorie (indicazione, pericolo, divieto, background).

2.1 Rete Utilizzata

In seguito a una fase preliminare di sperimentazione (discussa nel capitolo successivo), è stata selezionata una Convolutional Neural Network (CNN) ispirata ad AlexNet, denominata *LeNetColor*. L'architettura è stata progressivamente migliorata mediante l'introduzione di *Batch Normalization* e *Dropout*, al fine di stabilizzare l'addestramento e ridurre il rischio di overfitting. Nel seguito, il modello finale viene indicato come **MiniAlexNetV2**. La Figura 3 mostra la rete di partenza (*LeNetColor*) e la sua prima evoluzione (*MiniAlexNet*), mentre la Figura 4 riporta l'architettura finale selezionata. Per la configurazione di input sono state adottate scelte empiriche consolidate: batch size pari a 1024 in addestramento e 2048 in validazione/test, con strati di MaxPooling (kernel 2, stride 1). Come funzione di attivazione è stata utilizzata la Rectified Linear Unit (ReLU), che annulla i valori negativi e mantiene invariati i valori positivi. Tale funzione è descritta in (1):

$$f(x) = \begin{cases} 0 & \text{se } x < 0 \\ x & \text{se } x > 0 \end{cases} \quad (1)$$

2.2 Funzione di Loss

La funzione di loss adottata è la *Cross Entropy Loss*, riportata in (2):

$$\text{Cross Entropy Loss} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \cdot \log(\hat{y}_{ij}) \quad (2)$$

dove:

N è il numero di campioni,

C è il numero di classi,

y_{ij} è la vera probabilità della classe j per il campione i ,

$\hat{y}_{ij}$ è la probabilità predetta della classe j per il campione i .

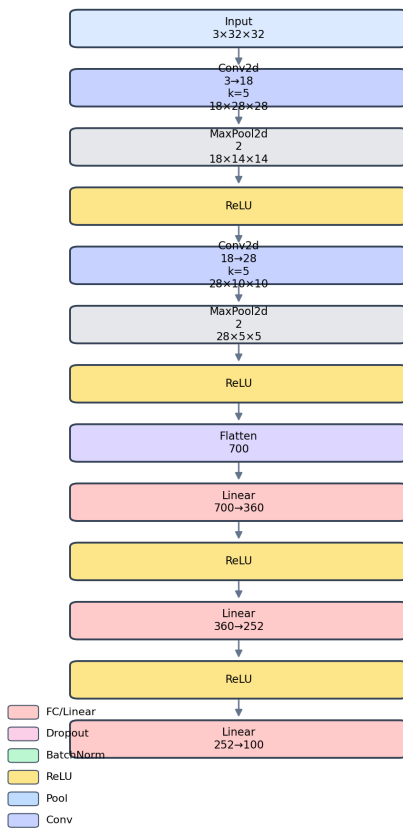
La cross entropy loss consente di confrontare due distribuzioni di probabilità: quella predetta dal modello e quella target. Nel caso in esame, **MiniAlexNetV2** produce per ciascuna immagine una distribuzione di probabilità sulle classi, che viene confrontata con la distribuzione reale.

In termini operativi, è stata inoltre considerata la seguente variante della funzione di loss:

$$\text{Cross Entropy Loss} = -\frac{1}{N} \sum_i^N (y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)) \quad (3)$$

LeNetColor

Schema architetturale verticale



MiniAlexNet

Schema architetturale verticale

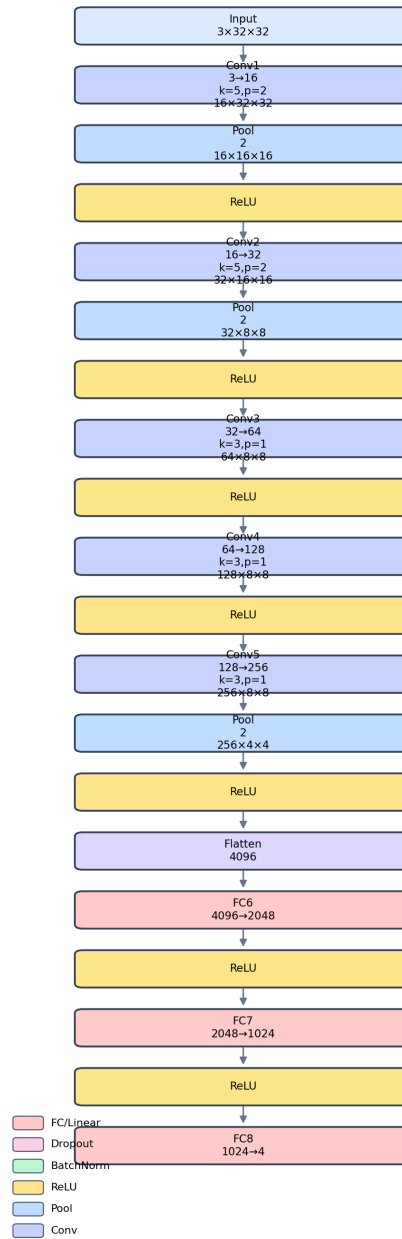


Figura 3: Confronto tra LeNetColor e MiniAlexNet

MiniAlexNetV2

Schema architetturale verticale

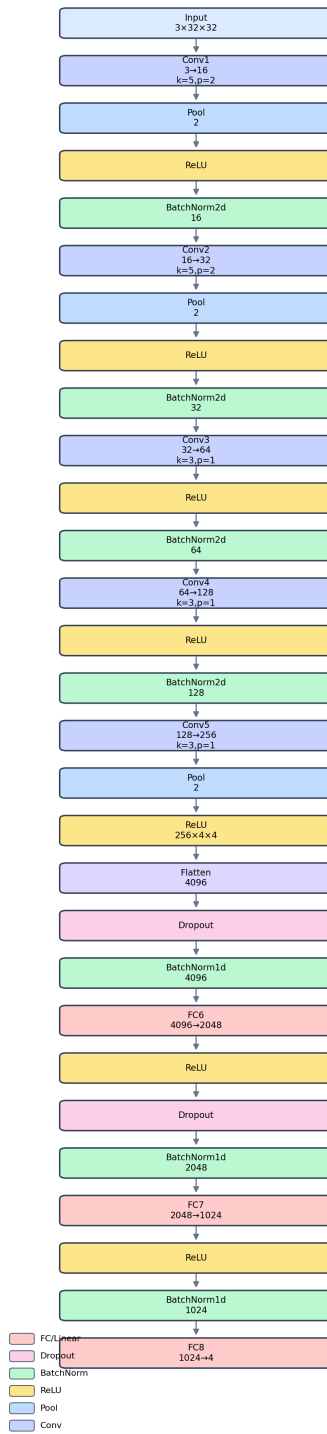


Figura 4: MiniAlexNetV2, rete utilizzata per il training

dove:

N è il numero di classi,

y_i è la vera probabilità della classe i ,

$\hat{y}_i$ è la probabilità predetta della classe i .

2.3 Valutazione

La qualità delle previsioni è stata valutata tramite il parametro di accuratezza, derivato a partire da una matrice di confusione elementare (Tabella 1):

	Realtà	
	Positivo	Negativo
Previsione Positiva	Vero Positivo	Falso Positivo
Previsione Negativa	Falso Negativo	Vero Negativo

Tabella 1: Matrice di Confusione

L'accuratezza del modello è definita come rapporto tra il numero di previsioni corrette e il numero totale di previsioni, come espresso in (4):

$$\text{Accuracy} = \frac{\text{Numero di Previsioni Corrette}}{\text{Numero Totale di Previsioni}} \quad (4)$$

Tale parametro fornisce una misura sintetica della capacità di classificazione del modello.

Accanto all'accuratezza è stata analizzata anche la curva ROC, che descrive l'andamento delle prestazioni al variare della soglia decisionale.

3 Analisi

Le analisi iniziali sono state finalizzate all'identificazione del modello più promettente. In una seconda fase è stato svolto lo studio delle configurazioni di iperparametri più efficaci, con l'obiettivo di massimizzare l'accuratezza del sistema.

3.1 Scelta del Modello

Per identificare il modello ottimale sono stati condotti diversi esperimenti comparativi. Nel primo è stata impiegata la rete **AlexNet**; nel secondo è stata testata una sua variante, **MiniAlexNet**; nel terzo, introducendo strati di *Dropout* e *Batch Normalization*, è stata ottenuta la versione **MiniAlexNetV2**.

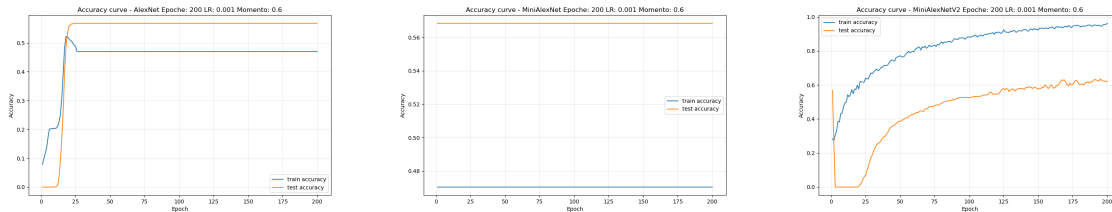


Figura 5: Confronto tra le curve di accuratezza delle tre reti (Learning Rate: 0.001, Momento: 0.6)

L'analisi della Figura 26 evidenzia differenze sostanziali tra le tre architetture. La rete **AlexNet** mostra un'accuratezza che si attesta intorno allo 0.3-0.4, con train accuracy leggermente superiore alla test accuracy, indicando un modesto overfitting. La rete **MiniAlexNet** presenta valori di accuratezza compresi tra 0.48 e 0.56, con train e test accuracy quasi sovrapposte, suggerendo una buona generalizzazione ma prestazioni limitate. La rete **MiniAlexNetV2** raggiunge invece accuratezza tra 0.6 e 0.8, con un gap tra train e test accuracy più marcato, indicativo di una maggiore capacità di apprendimento ma anche di un certo overfitting.

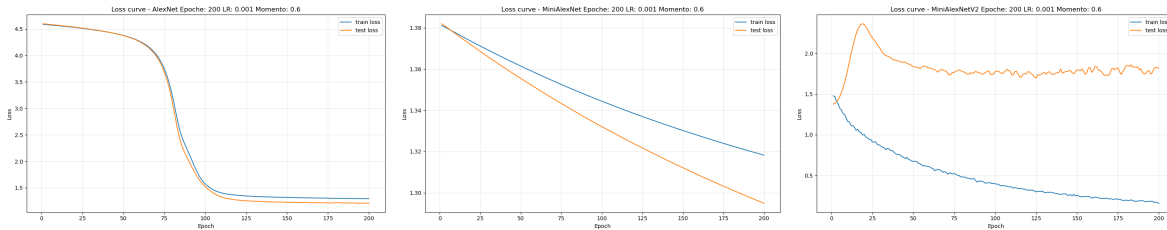


Figura 6: Confronto tra le curve di loss delle tre reti (Learning Rate: 0.001, Momento: 0.6)

Dall'analisi della Figura 27 emergono ulteriori elementi di differenziazione. La rete **AlexNet** presenta valori di loss tra 1.5 e 4.5, con una train loss in netta diminuzione mentre la test loss rimane elevata e instabile, segno di una capacità di generalizzazione molto limitata. La rete **MiniAlexNet** mostra loss più contenute e stabili, con train e test loss che convergono verso valori simili, riflettendo l'equilibrio tra apprendimento e generalizzazione osservato nelle curve di accuratezza. La rete **MiniAlexNetV2**

presenta la dinamica di apprendimento più efficace: nonostante un iniziale incremento della test loss nelle prime epoche, successivamente entrambe le curve si stabilizzano su valori contenuti intorno a 0.5-1.0, confermando la superiorità dell'architettura. I parametri utilizzati per queste prove sono stati:

- Tasso di apprendimento (*Learning Rate*): 0.001
- Momento (*Momentum*): 0.6

Complessivamente, tali risultati hanno motivato la prosecuzione dello studio sulla rete **MiniAlexNetV2**.

3.2 Curve ROC

Una volta individuate le tre configurazioni più significative, sono state calcolate le rispettive curve ROC, riportate in Figura 7. La curva ROC, che descrive la relazione tra tasso di veri positivi (TPR) e tasso di falsi positivi (FPR) al variare della soglia di classificazione, è stata utilizzata come criterio aggiuntivo per confrontare la capacità discriminante dei tre modelli.

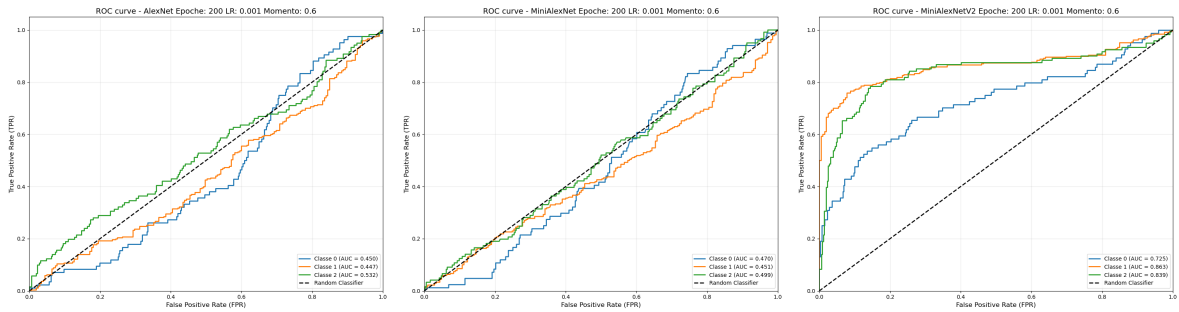


Figura 7: Confronto tra le ROC curve delle tre reti (Learning Rate: 0.001, Momento: 0.6)

Dall'analisi delle curve ROC in Figura 7 emerge un quadro chiaro delle prestazioni dei tre modelli. La rete **AlexNet** mostra un comportamento molto prossimo al caso casuale per tutte le classi, con curve che seguono la diagonale del classificatore random, indicando una capacità discriminante pressoché nulla. La rete **MiniAlexNet** presenta valori di TPR identici a quelli del classificatore casuale per ogni valore di FPR, suggerendo che il modello non è in grado di distinguere efficacemente tra le classi.

La rete **MiniAlexNetV2** si distingue nettamente dalle altre due: le curve TPR per le tre classi si discostano significativamente dalla diagonale casuale, con valori di TPR superiori al caso casuale per ogni livello di FPR. Le tre curve presentano un andamento crescente simile, indicando prestazioni bilanciate tra le classi, con le classi 0 e 1 che mostrano un leggero vantaggio rispetto alla classe 2. Questo comportamento conferma la superiorità dell'architettura MiniAlexNetV2 nell'estrarre caratteristiche discriminanti dai dati.

3.3 Calibrazione del Learning Rate e del Momento

Una volta selezionata MiniAlexNetV2 come architettura di riferimento, sono state condotte analisi approfondite per ottimizzare i iperparametri. Sono state testate diverse combinazioni di learning rate (0.0001, 0.001, 0.003) e momento (0.3, 0.6, 0.9), per un totale di nove configurazioni, al fine di identificare la combinazione ottimale per massimizzare l'accuratezza e la stabilità della convergenza. Al fine di evitare un'eccessiva frammentazione dell'esposizione grafica e di rendere più agevole il confronto tra i risultati, si è ritenuto opportuno non riportare l'intero insieme dei nove grafici relativi alle diverse combinazioni di iperparametri. Si è scelto, invece, di presentare esclusivamente il grafico corrispondente alla configurazione di learning rate e momentum adottata anche nelle analisi comparative con le altre architetture di rete, nonché quello relativo alla rete che ha evidenziato le prestazioni complessivamente migliori, insieme ai rispettivi iperparametri ottimali. I grafici saranno comunque disponibili nel capitolo **Appendice** per una possibile consultazione

3.3.1 Analisi dell'Accuratezza

La Figura 8 presenta le curve di accuratezza per le diverse combinazioni di iperparametri.

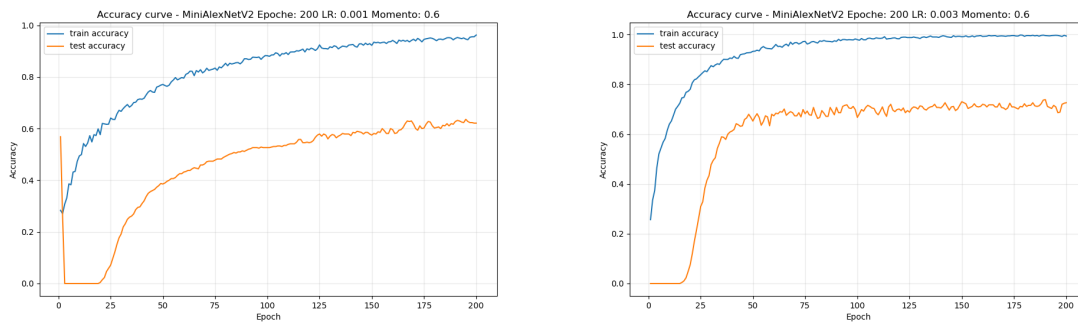


Figura 8: Confronto delle curve di accuratezza della MiniAlexNetV2

Dall'analisi delle curve di accuratezza emergono diverse osservazioni significative:

- **Learning Rate 0.0001:** Le configurazioni con learning rate ridotto mostrano un'accuratezza molto bassa, intorno a 0.1-0.2, per tutti i valori di momento. La rete non riesce ad apprendere efficacemente, con train e test accuracy che rimangono su livelli prossimi al caso casuale. L'unica eccezione è rappresentata dalla configurazione con momento 0.9, che mostra un leggero miglioramento, raggiungendo circa 0.4 di accuratezza verso la fine dell'addestramento.
- **Learning Rate 0.001:** Questa configurazione produce risultati intermedi. Con momento 0.3, l'accuratezza si attesta intorno a 0.4-0.5, mostrando un gap tra train e test accuracy indicativo di un moderato overfitting. Con momento 0.6, le prestazioni migliorano sensibilmente, raggiungendo 0.6-0.7 di accuratezza, con un comportamento più stabile. Con momento 0.9, si osserva un incremento fino a 0.7-0.8 di accuratezza, ma con fluttuazioni più marcate nella fase di test, suggerendo una maggiore sensibilità al momento elevato.

- **Learning Rate 0.003:** Le configurazioni con learning rate più elevato mostrano le prestazioni migliori. Con momento 0.3, l'accuratezza raggiunge 0.7-0.8, train e test accuracy convergono verso valori simili, indicando un buon equilibrio tra apprendimento e generalizzazione. Con momento 0.6, si osservano le prestazioni più elevate, con train accuracy che supera 0.9 e test accuracy che si attesta intorno a 0.8, train e test accuracy mostrano una buona convergenza. Con momento 0.9, l'accuratezza raggiunge valori simili a quelli della configurazione con momento 0.3, intorno a 0.7-0.8.

3.3.2 Analisi della Loss

La Figura 9 presenta le curve di loss per le diverse combinazioni di iperparametri, fornendo ulteriori informazioni sulla dinamica di apprendimento.

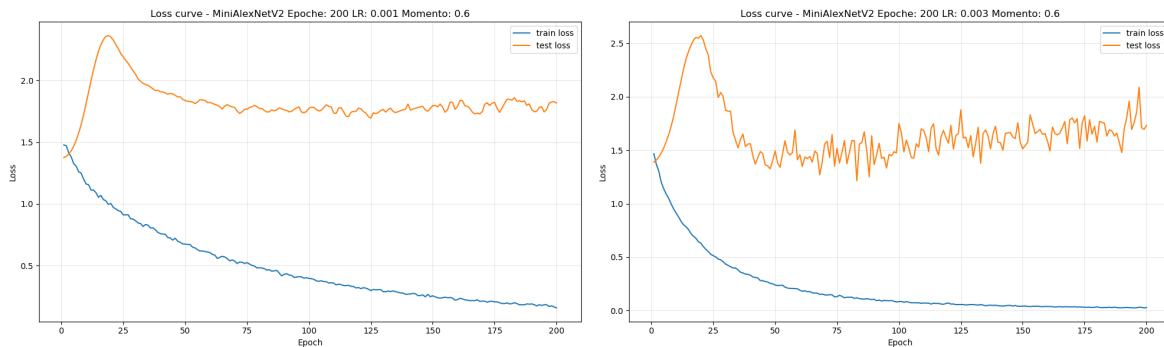


Figura 9: Confronto delle curve di loss per diverse della MiniAlexNetV2

Dall'analisi delle curve di loss emergono considerazioni che corroborano quanto osservato per l'accuratezza:

- **Learning Rate 0.0001:** Le loss rimangono elevate (1.1-1.7) per tutte le configurazioni, con train e test loss che convergono verso valori simili ma non riescono a diminuire significativamente. La rete non è in grado di ridurre l'errore, confermando l'insufficienza del learning rate per un apprendimento efficace.
- **Learning Rate 0.001:** Con momento 0.3, le loss scendono a valori intorno a 0.75-1.5, con un gap train-test che indica un overfitting contenuto. Con momento 0.6, le loss si riducono ulteriormente, train loss si avvicina a 0.5, mentre la test loss si attesta intorno a 1.0, train e test loss mostrano una dinamica di convergenza più stabile. Con momento 0.9, train loss scende a valori molto bassi (intorno a 0.25), ma test loss rimane elevata (1.0-1.5), indicando un significativo overfitting.
- **Learning Rate 0.003:** Con momento 0.3, train e test loss decrescono rapidamente nelle prime epoche, con valori di train loss intorno a 1.0 e test loss intorno a 1.5, mostrando una buona capacità di apprendimento. Con momento 0.6, si osserva la convergenza migliore: train loss scende fino a 0.5 e test loss intorno a 0.8-1.0, con un gap contenuto. Con momento 0.9, train loss scende rapidamente verso 0.25, mentre test loss si stabilizza intorno a 1.0, evidenziando un overfitting più marcato rispetto alla configurazione con momento 0.6.

3.3.3 Curve ROC per le Diverse Configurazioni

Per valutare più approfonditamente la capacità discriminante delle diverse configurazioni, sono state analizzate le curve ROC per ciascuna combinazione di iperparametri.

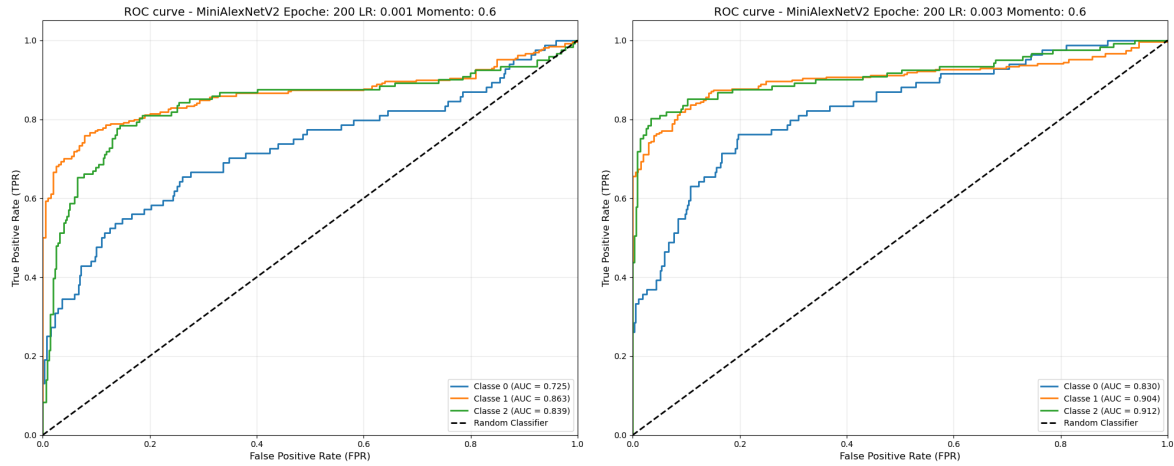


Figura 10: Confronto delle ROC curve per diverse combinazioni della MiniAlexNetV2

L'analisi delle curve ROC in Figura 10 fornisce ulteriori elementi per la selezione della configurazione ottimale:

- **Learning Rate 0.0001:** Le curve ROC mostrano un comportamento molto vicino al caso casuale per tutte le classi, con valori di TPR che seguono la diagonale del classificatore random. La capacità discriminante è minima per tutte le configurazioni, con un leggero miglioramento per momento 0.9, dove le curve iniziano a discostarsi leggermente dalla diagonale.
- **Learning Rate 0.001:** Con momento 0.3, le curve ROC si discostano dalla diagonale ma in modo limitato, con TPR che raggiunge circa 0.2 per FPR di 0.1. Con momento 0.6, si osserva un miglioramento significativo, con TPR che supera 0.1 per FPR vicino a 0.02. Con momento 0.9, le prestazioni migliorano ulteriormente, con TPR che raggiunge 0.5 per FPR di 0.1 per le classi 1 e 2, indicando una buona capacità discriminante, anche se la classe 0 mostra prestazioni inferiori.
- **Learning Rate 0.003:** Questa configurazione produce le migliori curve ROC. Con momento 0.3, le curve si discostano significativamente dalla diagonale, con TPR che supera 0.6 per FPR di 0.3. Con momento 0.6, si osservano le migliori prestazioni, con TPR che raggiunge 1.0 per FPR di circa 0.2 per la classe 2, e valori elevati per le altre classi, dimostrando una capacità discriminante molto buona. Con momento 0.9, le curve mostrano un comportamento anomalo con TPR che supera 1.0 per alti valori di FPR, probabilmente dovuto a instabilità nell'addestramento o a problemi nei dati, suggerendo che questo valore di momento non è ottimale.

Tabella 2: Configurazione Ottimale Identificata

Parametro	Valore
Learning Rate	0.003
Momento	0.6
Numero di epoche	200 (con possibilità di estendere a 1200)

3.3.4 Analisi dell'Addestramento Prolungato

Per valutare il comportamento del modello su un numero maggiore di epoche, è stata condotta una prova con la configurazione che aveva mostrato le prestazioni più promettenti (Learning Rate 0.003, Momento 0.6) per 1200 epoche.

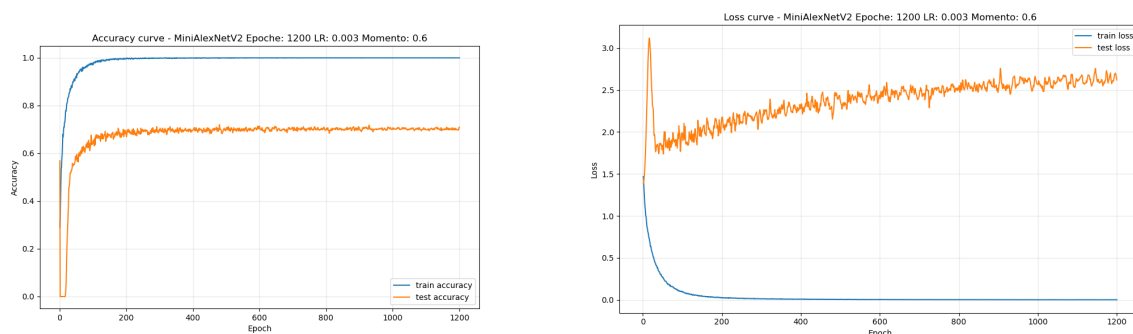


Figura 11: Curve di accuratezza e loss per MiniAlexNetV2 con Learning Rate 0.003, Momento 0.6 su 1200 epoche

Dall'analisi della Figura 11 emergono considerazioni importanti:

- **Accuratezza:** L'accuratezza di train raggiunge rapidamente valori prossimi a 1.0 già dopo poche centinaia di epoche, mentre l'accuratezza di test si stabilizza intorno a 0.85-0.90. Il gap tra train e test accuracy rimane costante nel tempo, indicando un overfitting che non si riduce con l'aumentare delle epoche.
- **Loss:** La train loss converge a valori prossimi a 0.0 dopo circa 400-500 epoche, mentre la test loss si stabilizza intorno a 0.5-1.0. L'andamento mostra una leggera tendenza all'aumento della test loss nelle fasi avanzate, suggerendo un possibile deterioramento della capacità di generalizzazione.
- **Curve ROC:** Le curve ROC per l'addestramento prolungato mostrano una capacità discriminante molto elevata, con TPR che raggiunge 1.0 per tutte le classi già a FPR di 0.2-0.3, indicando un'ottima separazione delle classi. Le prestazioni sono bilanciate tra le tre classi, con lievi differenze a favore della classe 1.

In sintesi quello che si può dire è che l'allenamento a 1200 epoche è stato utile ma superfluo anche perchè si sarebbe potuto raggiungere il medesimo risultato per 400-500 epoche.

3.3.5 Sintesi dei Risultati e Configurazione Ottimale

Sulla base dell'analisi complessiva delle diverse combinazioni di iperparametri, è possibile trarre le seguenti conclusioni:

- Il **learning rate 0.0001** si è rivelato insufficiente per un apprendimento efficace, indipendentemente dal valore del momento. La rete non riesce a ridurre significativamente la loss né a migliorare l'accuratezza oltre il caso casuale.
- Il **learning rate 0.001** produce risultati intermedi, con un momento di 0.9 che offre le prestazioni migliori in termini di accuratezza e capacità discriminante, sebbene con un overfitting più marcato.
- Il **learning rate 0.003** si è dimostrato il più efficace, in particolare con un momento di 0.6. Questa combinazione offre il miglior compromesso tra accuratezza, capacità discriminante (curve ROC nettamente superiori al caso casuale) e stabilità della convergenza.
- Il **momento 0.9**, sebbene possa portare a un apprendimento più rapido, tende a causare overfitting e instabilità, come evidenziato dalle curve ROC anomale e dal gap train-test nelle curve di loss.

Pertanto, la configurazione ottimale individuata per il modello MiniAlexNetV2 è:

- **Learning Rate:** 0.003
- **Momento:** 0.6
- **Numero di epoche:** 200 (con possibilità di estendere a 1200 per un miglioramento marginale)

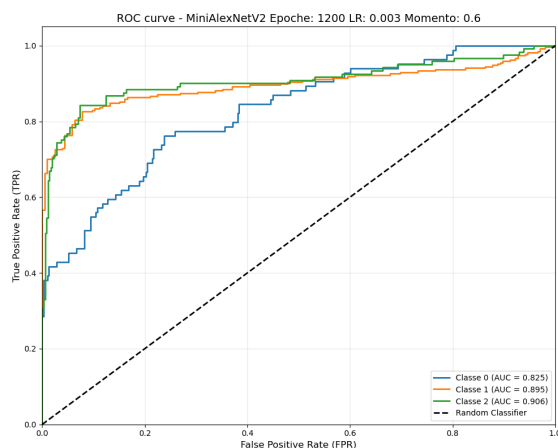


Figura 12: Curve ROC per MiniAlexNetV2 con Learning Rate 0.003, Momento 0.6 su 1200 epoche

L'addestramento prolungato a 1200 epoche, pur non migliorando significativamente le prestazioni rispetto a 200 epoche, conferma la robustezza della configurazione selezionata e fornisce indicazioni sulla stabilità a lungo termine del modello.

4 Codice

Il codice del progetto è stato organizzato nella sottocartella `python_file`, che contiene i seguenti moduli:

- Il file `dataclass` definisce le classi e le trasformazioni per la gestione del dataset di segnali stradali. In particolare, include le classi per l'addestramento (`StreetSignDataset`) e per il test (`StreetSignTest`), oltre a trasformazioni quali ridimensionamento (`Rescale`), ritaglio casuale (`RandomCrop`), ritaglio specifico (`Crop`) e conversione in tensori (`ToTensor`). È inoltre presente la funzione `show_landmarks_batch`, utilizzata per visualizzare batch di immagini con i relativi *landmarks*. Nel complesso, il modulo fornisce l'infrastruttura necessaria alla preparazione dei dati.
- Il file `function` raccoglie le funzioni principali per addestramento e test del classificatore. La prima parte del codice esegue l'addestramento mediante *Stochastic Gradient Descent* e *Cross Entropy Loss*, con supporto a TensorBoard e salvataggio dei pesi. La seconda parte valuta il modello sul set di test, restituendo predizioni ed etichette reali. Nello stesso file sono inoltre presenti le funzioni per l'esportazione dei risultati in formato JSON/CSV e per la generazione dei grafici di accuratezza e loss. È infine inclusa una funzione per il calcolo e la visualizzazione della curva ROC a partire da etichette e predizioni.
- Il file `network` implementa le architetture di rete neurale testate, includendo diverse varianti e l'impiego di tecniche quali *Batch Normalization* e *Dropout*, utili a migliorare prestazioni e capacità di generalizzazione.
- Il file `detect_and_classify` implementa la procedura per applicare YOLO al riconoscimento dei segnali stradali in immagini di scenario reale. L'immagine viene inizialmente analizzata da YOLO (addestrato su immagini reali) per individuare il segnale; la porzione ritagliata viene quindi fornita alla rete **MiniAlexNetV2** per la classificazione. Il sistema realizza così una pipeline completa di *detection* e *classification*.

Infine, sono stati creati alcuni notebook in Jupyter, nei quali è stata utilizzata la struttura dati definita in precedenza insieme alle funzioni di training e test, al fine di addestrare il modello. Prima dell'addestramento, le immagini vengono convertite nel formato richiesto dalla struttura dati e sottoposte alle opportune trasformazioni. I notebook realizzati sono i seguenti:

- I notebook con intestazione *train* si occupano dell'addestramento delle reti *AlexNet*, *MiniAlexNet* e *MiniAlexNetV2*. In particolare, il notebook esegue il caricamento delle immagini, la loro conversione in tensori e l'addestramento dei modelli, salvando i dati di accuratezza e loss per ogni epoca. Al termine dell'addestramento, i pesi dei modelli vengono salvati per essere successivamente utilizzati nei notebook di *test*, *result* e *graphs*.
- *graphs*: carica i dati di accuratezza e loss salvati nei notebook di addestramento e genera i grafici corrispondenti (Loss, Accuracy e ROC).

- *result*: carica i tre modelli precedentemente addestrati ed esegue una previsione a partire da un'immagine fornita tramite URL oppure da un'immagine locale. Il notebook restituisce quindi la classe predetta del segnale stradale, tra cui *indicatory* (indicazione), *prohibitory* (divieto) e *warning* (pericolo). In caso di errore nella previsione, viene visualizzato il messaggio *ERROR 404 NOT FOUND*. La classe *Background*, sebbene presente nella fase di addestramento, non viene considerata in quanto non rilevante ai fini dell'applicazione.
- *test_reale*: carica esclusivamente il modello *MiniAlexNetV2* e lo utilizza per eseguire la previsione della categoria del segnale stradale presente in un'immagine reale. In questo caso, l'immagine viene prima elaborata per individuare e ritagliare il segnale stradale, che viene successivamente passato alla rete *MiniAlexNetV2* per la classificazione. La classe predetta viene quindi sovrapposta all'immagine, che viene infine visualizzata.

4.1 Caso Reale

Analizziamo più in dettaglio il notebook *test_reale* in cui è presente la parte di codice relativa al test della rete neurale in un contesto urbano. Se, per esempio, dotassimo un veicolo di una telecamera per riprendere la strada in tempo reale, potremmo acquisire delle immagini durante il tragitto e verificare la presenza di segnali stradali, inviando infine l'output al sistema di bordo o al guidatore. Per fare ciò, ci si è posti il problema di come isolare la singola immagine del segnale rispetto al contesto. Per eseguire questa ricerca sono stati studiati due processi:

- **Divisione in sottoimmagini**: consiste nel suddividere l'immagine in numerose porzioni e verificare per ognuna di esse se contiene un segnale stradale.
- **Uso di YOLO**: utilizzo di uno strumento di object detection che permette di individuare le regioni d'interesse (ROI) all'interno delle immagini, identificando le porzioni candidate alla classificazione.

4.1.1 Divisione

L'uso della divisione in sottoimmagini è la soluzione più semplice, ma ha presentato scarsi risultati sia in termini di prestazioni che di affidabilità. Il motivo risiede nella necessità di processare sequenzialmente ogni ritaglio, causando lentezza di calcolo. Inoltre, l'accuratezza è limitata dal fatto che il dataset di addestramento non copre tutte le possibili variazioni del contesto circostante. Si aggiunge poi il problema del ritaglio: quando si esegue il taglio dell'immagine, il segnale non sempre ricade al centro della sottoimmagine e può risultare solo parziale. Questo fa sì che la rete non sia in grado di riconoscere il segnale oppure produca falsi positivi.

4.1.2 YOLO

Per ovviare ai limiti della suddivisione manuale, nel notebook *test_reale* è stato implementato l'uso dell'algoritmo **YOLO** (You Only Look Once). A differenza dell'approccio precedente, YOLO esegue l'object detection in un unico passaggio, identificando istantaneamente le coordinate dei segnali stradali (*Bounding Boxes*) all'interno di scene

complesse, per poi passare la porzione d'immagine ritagliata contenente il segnale alla rete *MiniAlexNetV2* per identificarne la tipologia.

Il flusso di lavoro implementato prevede i seguenti passaggi:

1. Caricamento dell'immagine ad alta risoluzione catturata nel contesto urbano.
2. Applicazione del modello YOLO per individuare le regioni d'interesse (ROI) corrispondenti ai segnali.
3. Ritaglio (*cropping*) automatico delle aree individuate.
4. Passaggio dei ritagli alla rete **MiniAlexNetV2** per la classificazione finale della macro-categoria.

Questo approccio ibrido ha permesso di ottenere una precisione estremamente elevata, poiché la rete di classificazione lavora solo su porzioni di immagine già identificate come "segnali", eliminando il rumore di fondo del contesto cittadino e riducendo drasticamente i tempi di calcolo rispetto alla scansione sequenziale.

4.1.3 Analisi dei Risultati

Una volta spiegati i vari metodi, si procede a visualizzare la differenza tra i due approcci utilizzati. Prima nel caso di utilizzo di un'immagine presa dalle immagini di test (Figura 13) con cui abbiamo addestrato la rete: possiamo notare che, nel caso dell'algoritmo di **divisione**, abbiamo ottenuto dei falsi positivi e, inoltre, alcune porzioni di immagine, essendo state ritagliate in modo non perfetto, sono state classificate erroneamente. Per quanto riguarda l'uso di **YOLO**, invece, si è avuta l'identificazione di un solo segnale, che è stato poi classificato correttamente dalla rete *MiniAlexNetV2*, ma non ha identificato gli altri segnali presenti.

Per quanto riguarda invece il caso in cui viene usata un'immagine campione casuale presa online (Figura 14), possiamo notare che **YOLO** si è comportato meglio rispetto all'algoritmo di **divisione**, poiché quest'ultimo ha eseguito dei tagli che talvolta risultavano superflui.

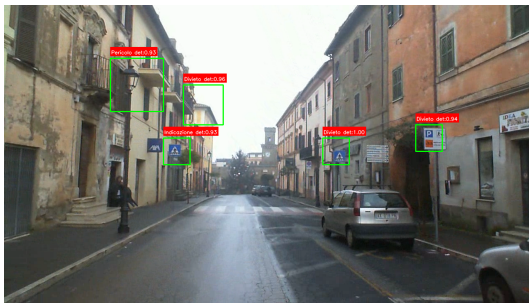
In sintesi, nessuno dei due metodi si è rivelato perfetto nell'analisi di un contesto reale, ma entrambi possono essere utilizzati come punto di partenza per comprendere come procedere.



(a) Immagine originale



(b) Immagine originale con label



(c) Divisione

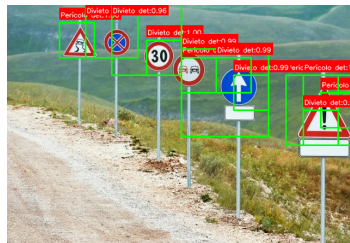


(d) YOLO

Figura 13: Confronto tra i metodi di divisione e YOLO su un'immagine del dataset di test.



(a) Immagine originale



(b) Divisione



(c) YOLO

Figura 14: Confronto tra i metodi di divisione e YOLO su un'immagine casuale.

5 Conclusione

I risultati ottenuti indicano che il modello proposto è in grado di affrontare efficacemente il compito assegnato, pur lasciando spazio a ulteriori miglioramenti. In prospettiva, si suggerisce di:

- Aumentare quantità e diversità dei dati di addestramento, così da migliorare la capacità di generalizzazione della rete.
- Sperimentare architetture neurali alternative, al fine di individuare quella più adatta al riconoscimento dei segnali stradali.
- Rafforzare l'uso di tecniche di regolarizzazione per ridurre l'*overfitting* e limitare l'eccessivo adattamento ai dati di addestramento.
- Ottimizzare gli *iperparametri* (ad esempio tasso di apprendimento e dimensione del batch) per massimizzare le prestazioni del modello.
- Valutare la quantizzazione dei pesi per ridurre la complessità del modello mantenendo un livello di accuratezza adeguato.
- Analizzare le attivazioni nei diversi strati della rete per approfondire la comprensione del processo di apprendimento.

In sintesi, il lavoro svolto costituisce una base solida per sviluppi successivi nel riconoscimento dei segnali stradali mediante modelli di *machine learning*. L'iterazione sperimentale e l'esplorazione di nuove soluzioni restano elementi centrali per affrontare scenari sempre più complessi. Una possibile evoluzione applicativa consiste nel passaggio dalla classificazione per macrocategorie all'identificazione del singolo segnale.

Appendice

Il codice sorgente del progetto è disponibile al seguente link: https://github.com/gorgons97/Reti_Neurali_per_Signali_detection. Il repository contiene tutti i file necessari per replicare l'addestramento e il test delle reti neurali, inclusi i notebook Jupyter, i moduli Python e i dati di addestramento. Gli utenti interessati possono scaricare il codice e i dati, eseguire gli script e analizzare i risultati ottenuti.

Il presente capitolo include le figure relative alle curve di accuratezza, loss e ROC delle reti neurali addestrate, come descritto nella sezione *Analisi dei Risultati*. Le immagini riportate forniscono una rappresentazione visiva delle prestazioni dei modelli durante il processo di addestramento e test.

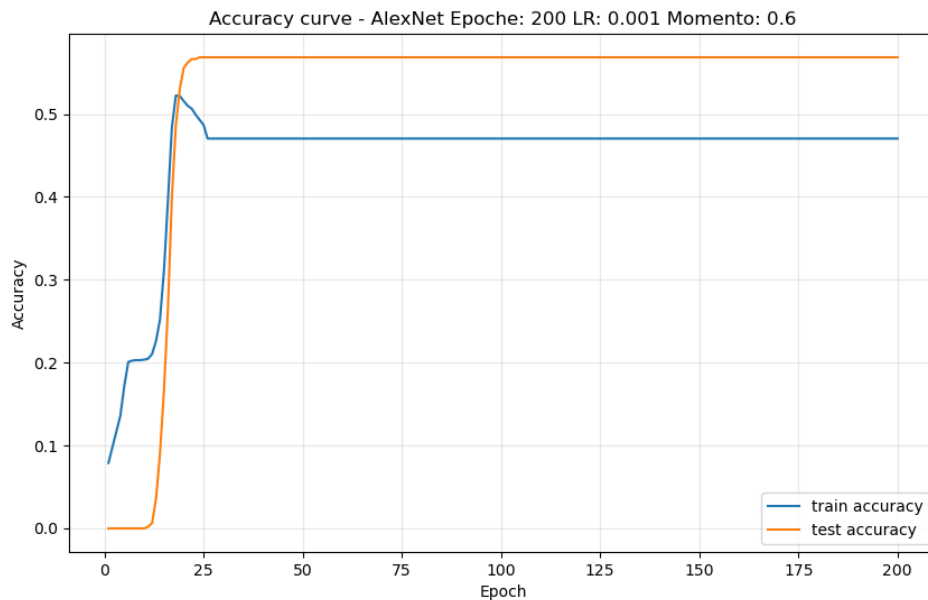


Figura 15: Curva di accuratezza della rete AlexNet.

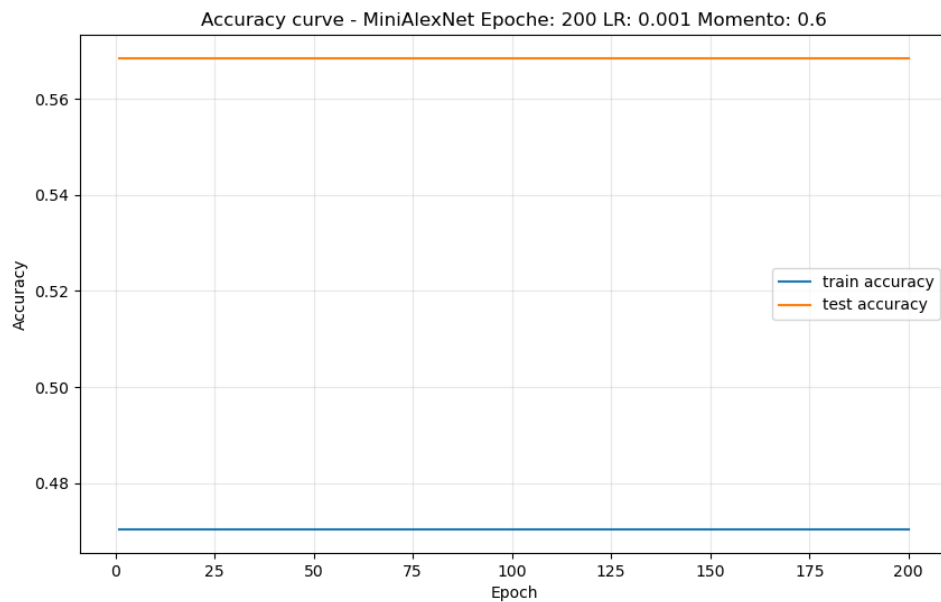


Figura 16: Curva di accuratezza della rete MiniAlexNet.

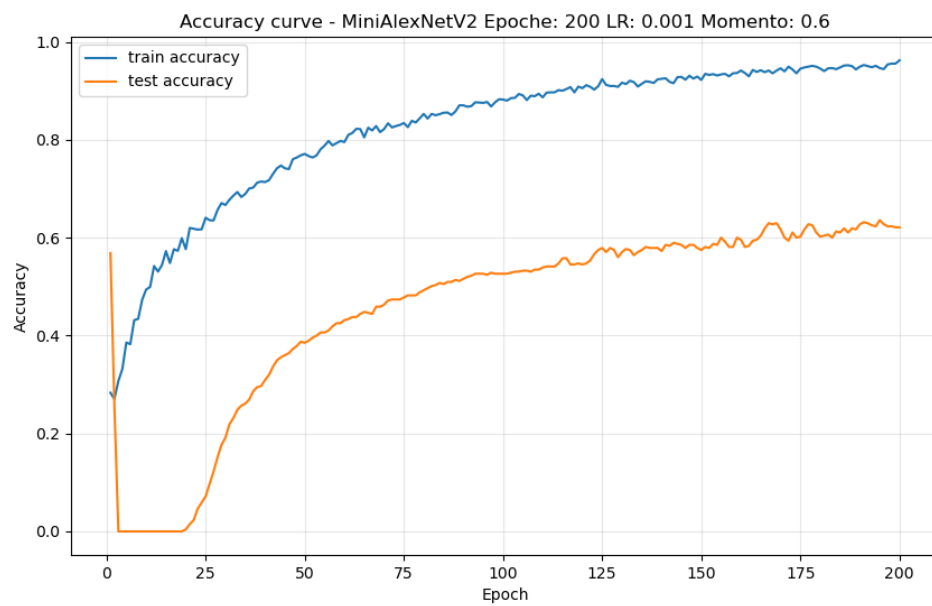


Figura 17: Curva di accuratezza della rete MiniAlexNetV2.

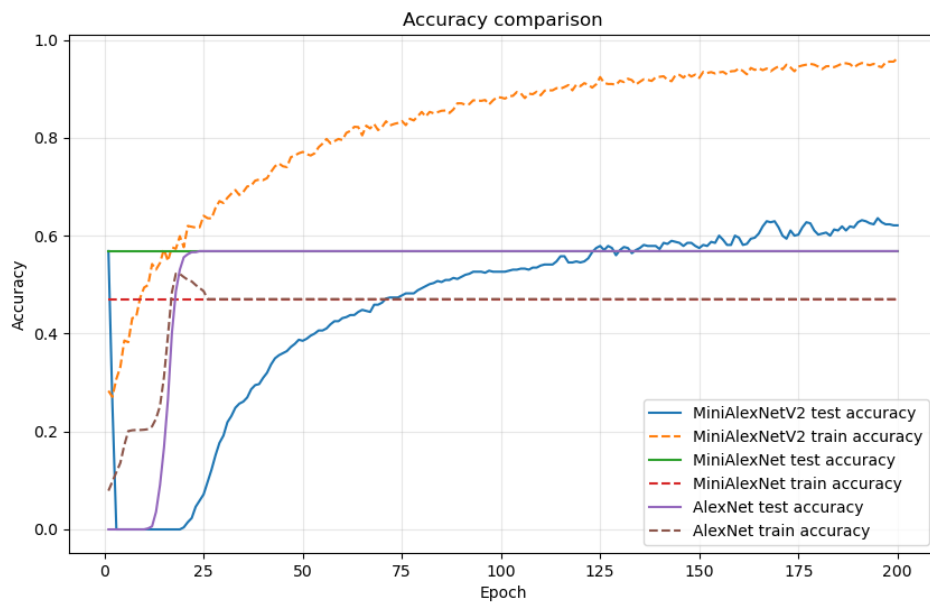


Figura 18: Curve di accuratezza delle reti.

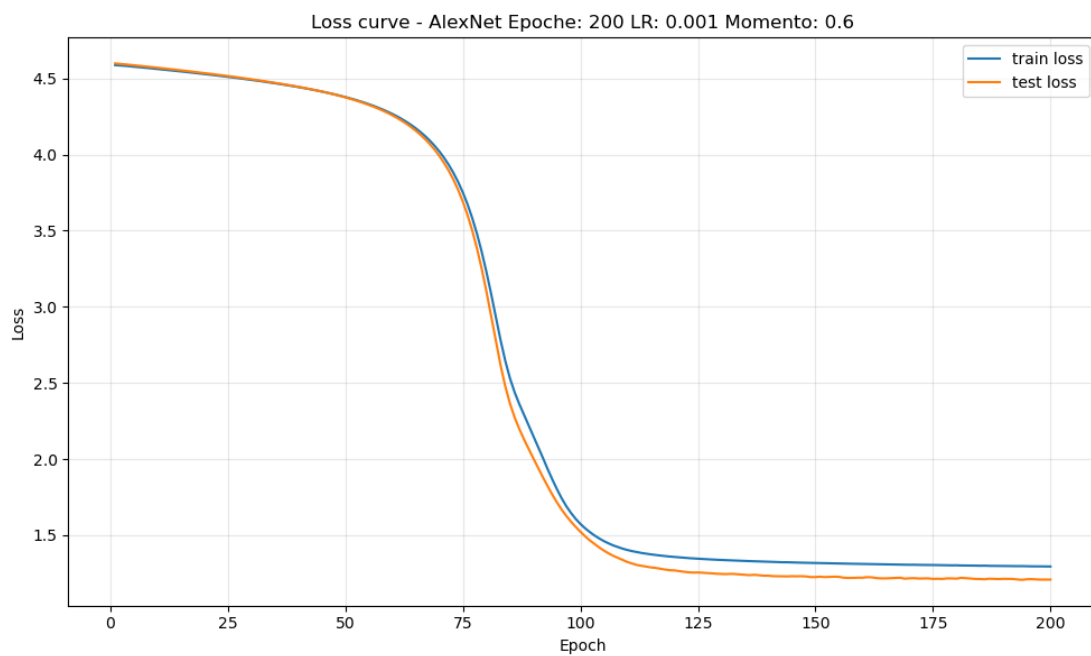


Figura 19: Curva di loss della rete AlexNet.

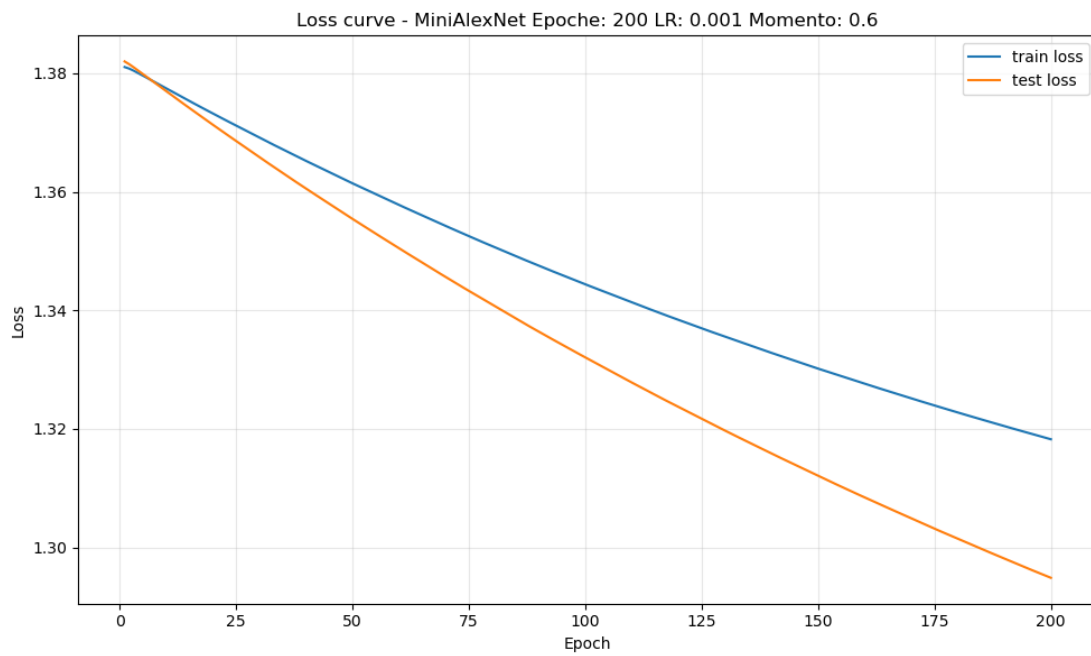


Figura 20: Curva di loss della rete MiniAlexNet.

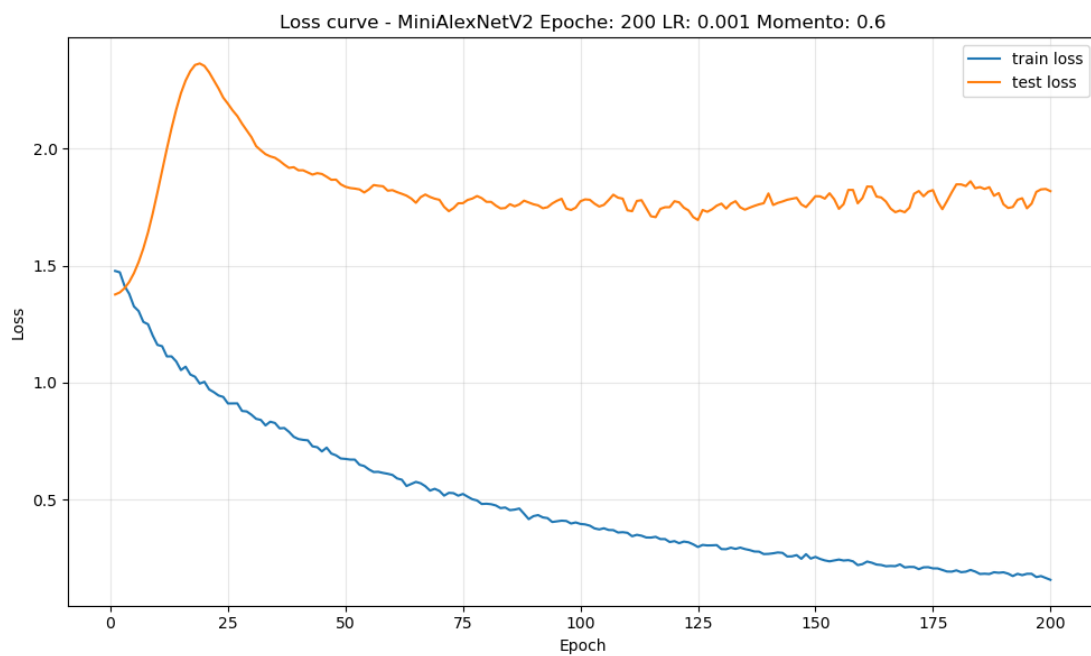


Figura 21: Curva di loss della rete MiniAlexNetV2.

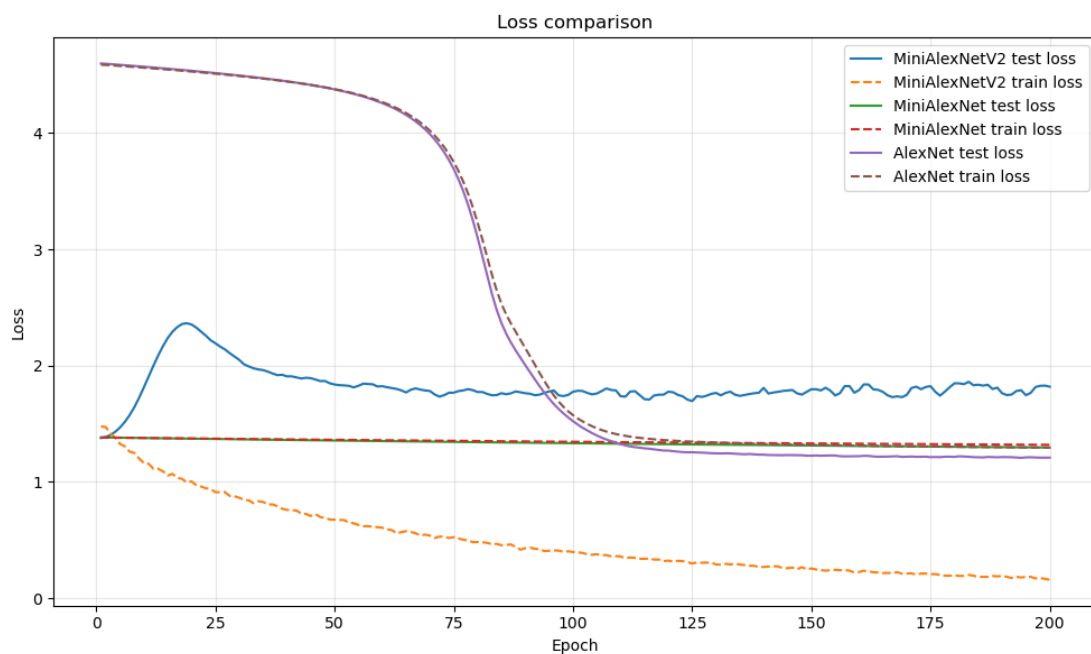


Figura 22: Curve di loss delle reti.

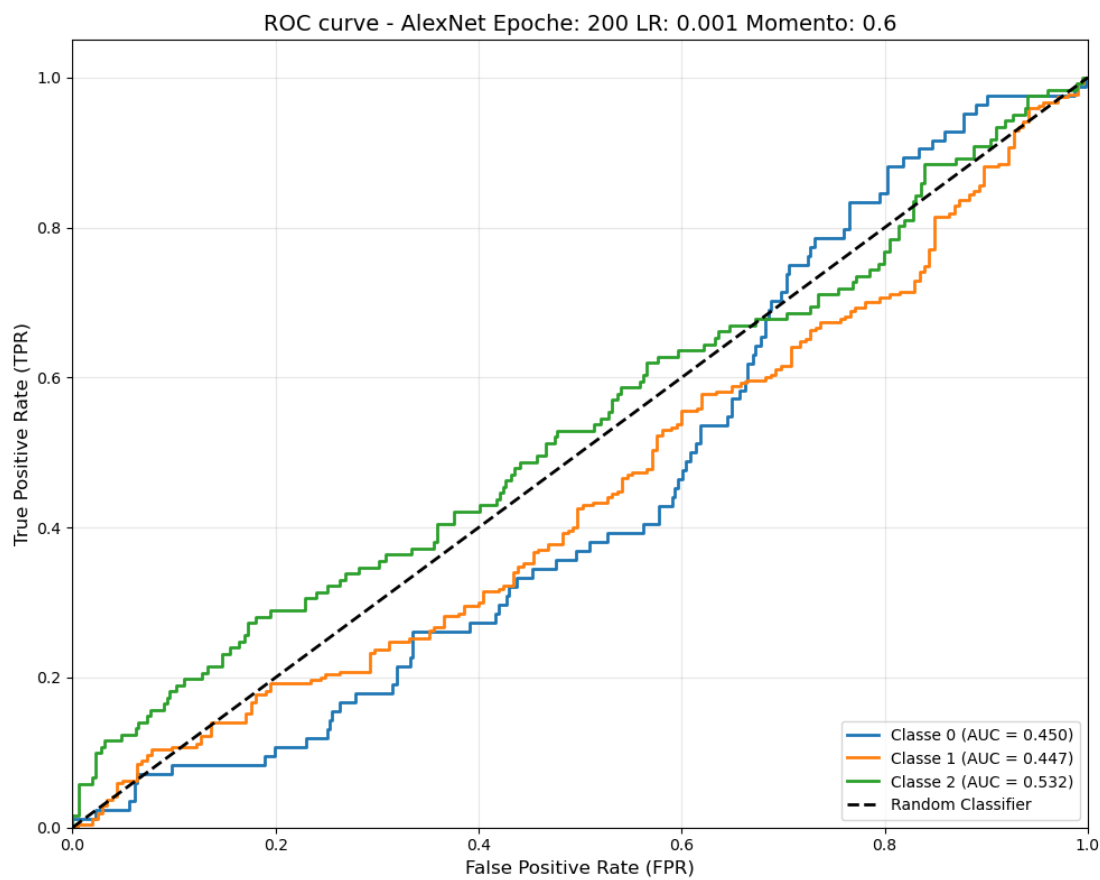


Figura 23: Curve ROC della rete AlexNet.

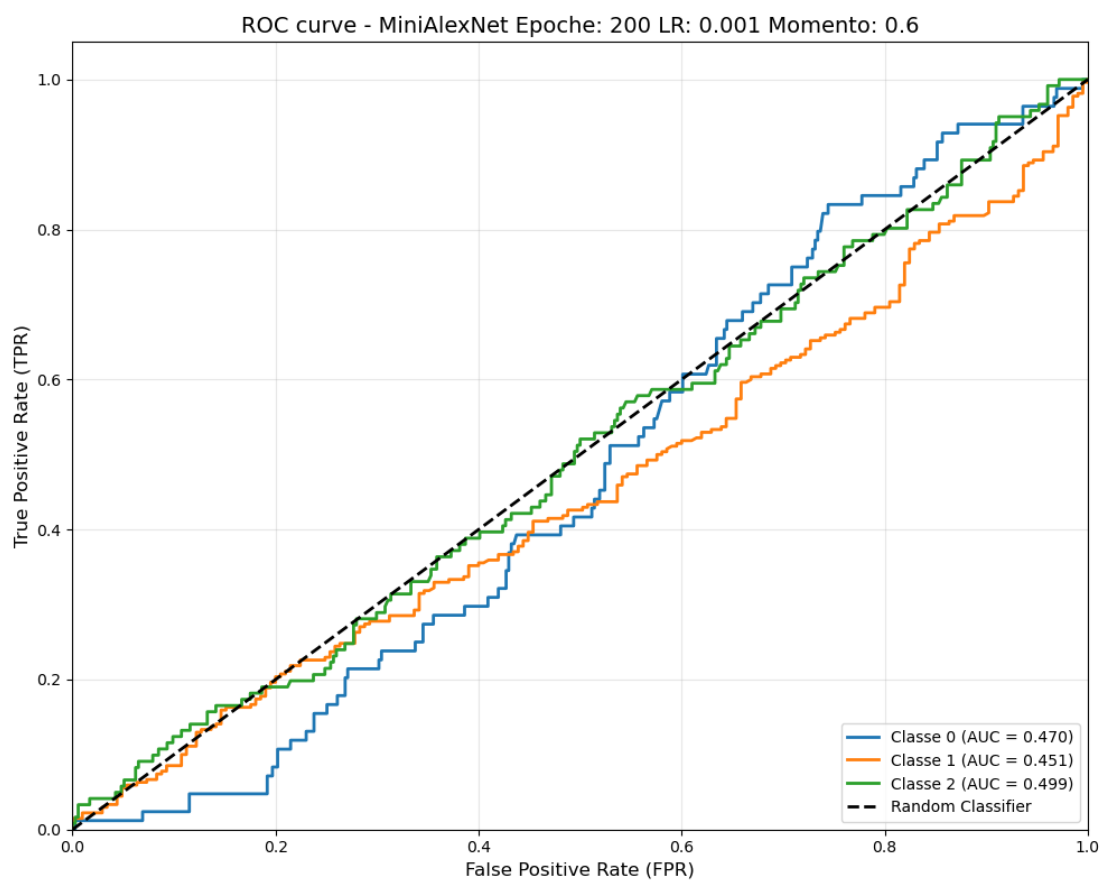


Figura 24: Curve ROC della rete MiniAlexNet.

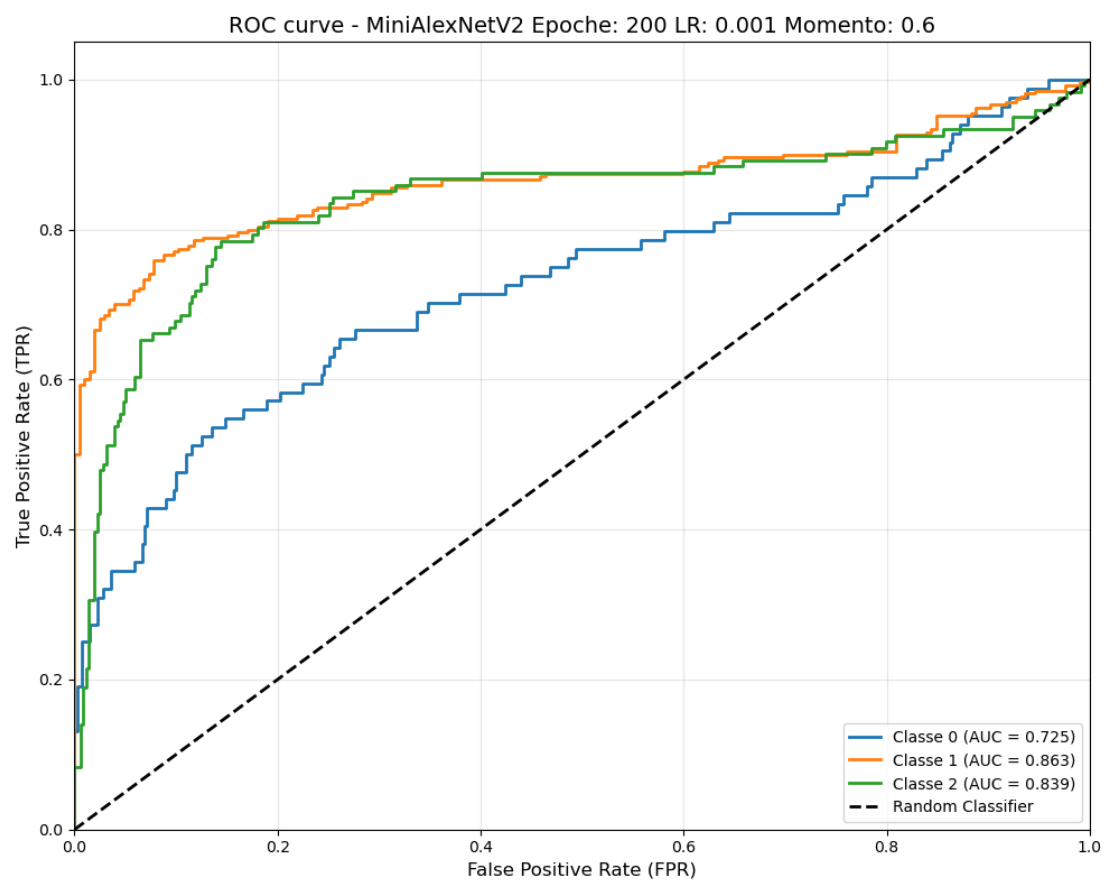


Figura 25: Curve ROC della rete MiniAlexNetV2.

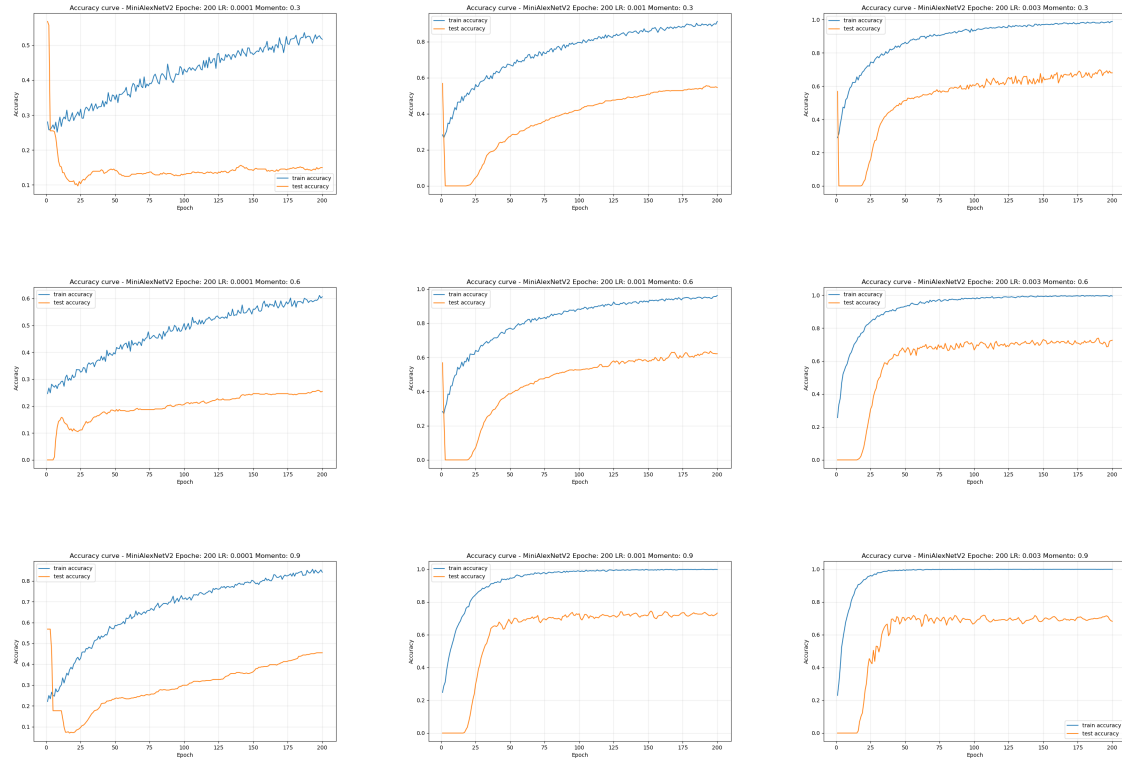


Figura 26: Confronto delle curve di accuratezza per diverse combinazioni di Learning Rate e Momento su MiniAlexNetV2

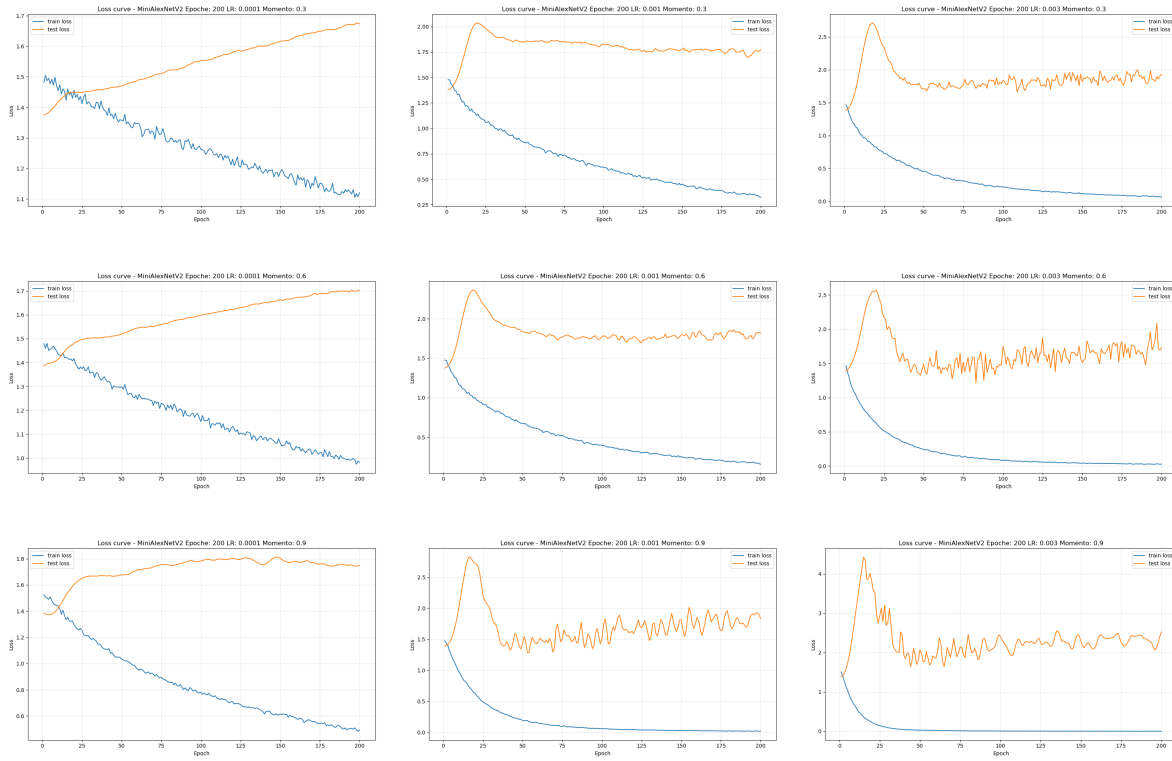


Figura 27: Confronto delle curve di loss per diverse combinazioni di Learning Rate e Momento su MiniAlexNetV2

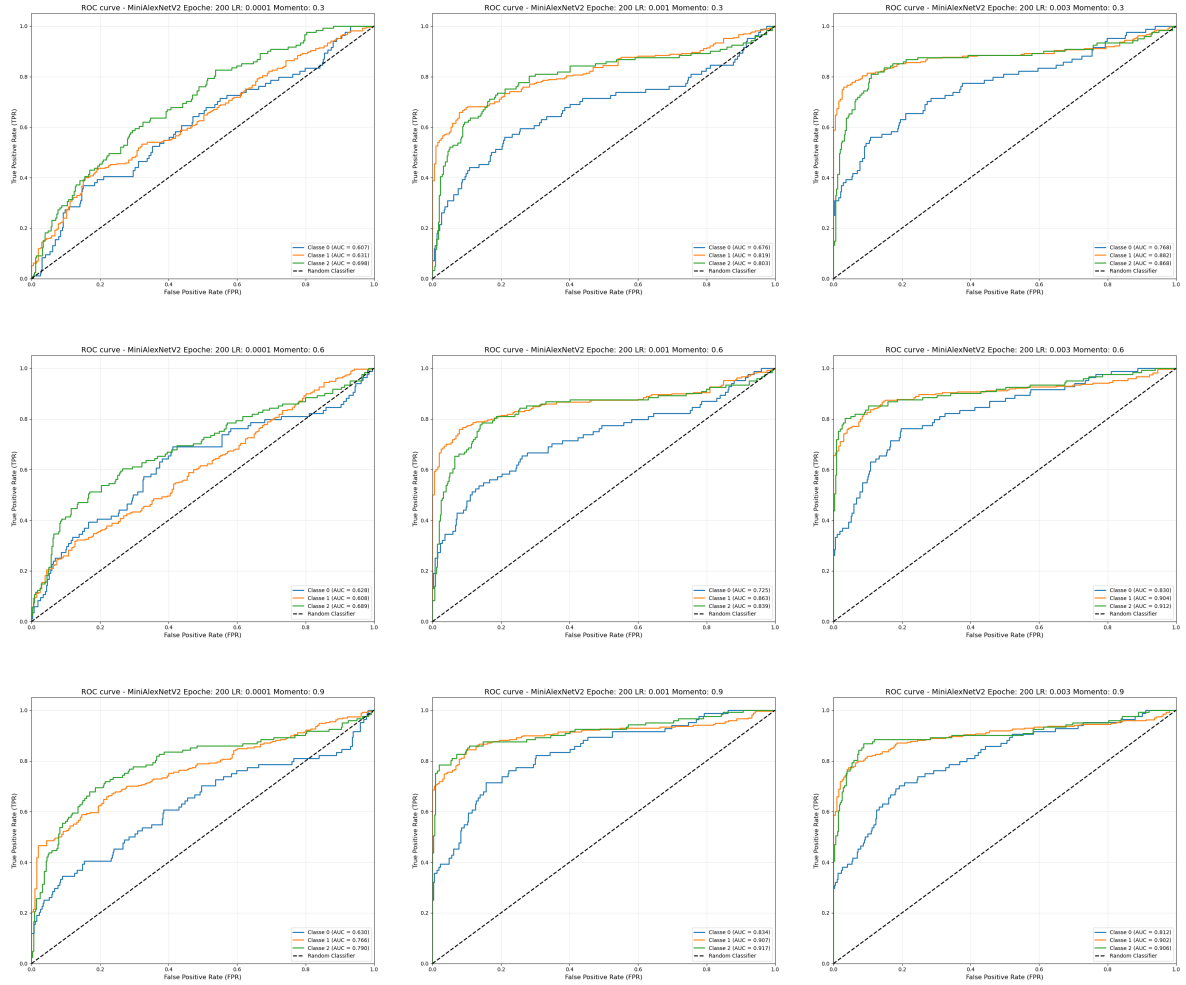


Figura 28: Confronto delle ROC curve per diverse combinazioni di Learning Rate e Momento su MiniAlexNetV2